Python task questions

Question 1.

Build a menu driven calculator using if-elif.

Question 2.

Create login system with username and password validation.

Question 3.

Find GCD and LCM of two numbers.

Question 4.

*

***

*****

****

Question 5.

Check whether two strings are anagrams. Anagrams means two or more strings that contain the exact same characters with the same frequency, but usually in a different order.

Question 6.

Create matrix addition program using nested lists

Question 7.

Create student record system using dictionary.

Question 8.

A robotic arm receives joint angles:

[30, -15, 45, 200, 60, 90]
Write a function to identify valid joint angles (0° to 180°) and use filter() to remove invalid angles. Then write another function to convert the valid angles into servo motor commands (angle × 10) using map().
Expected Output:
Valid Angles: [30, 45, 60, 90]
Servo Commands: [300, 450, 600, 900]
Question 9.
Robot Vision Detection System

A robot vision sensor captures object detection data as:

```
detections = [
    {"object": "box", "confidence": 78, "mode": "infrared", "distance": 2.5},
    {"object": "human", "confidence": 95, "mode": "camera", "distance": 1.2},
    {"object": "ball", "confidence": 82, "mode": "ultrasonic", "distance": 3.0},
    {"object": "human", "confidence": 88, "mode": "camera", "distance": 0.8},
    {"object": "chair", "confidence": 70, "mode": "infrared", "distance": 2.8}
```

]

Use filter() with lambda to keep only valid human detections captured through camera mode with confidence > 85%, use map() with lambda to extract their distances, and write a function to print "ALERT: Human very close!" if distance is < 1 meter, otherwise "Human detected at safe distance".

Expected Output:

Valid Human Detections:
[{'object': 'human', 'confidence': 95, 'mode': 'camera', 'distance': 1.2},
 {'object': 'human', 'confidence': 88, 'mode': 'camera', 'distance': 0.8}]

Distances:
[1.2, 0.8]

Human detected at safe distance
ALERT: Human very close!
Question 10.
Line Following Robot.
A 6-channel IR sensor in a line-following robot

gives input where:

1 = Line detected
0 = No line detected

Example input:
001100

Create a Python package named line_follower_robot with:

```
line_follower_robot/
  __init__.py
  sensor.py
  movement.py
```

Tasks:

1. In sensor.py:
Take user input for 6 sensor values.

Use map() to convert the input into integers.
Use filter() to count how many sensors detected the line.

2. In movement.py: Create a function to decide

robot movement:

If middle sensors detect line → Move Forward
If left sensors detect line → Turn Left
If right sensors detect line → Turn Right

If all sensors are 0 → Stop Robot
If all sensors are 1 → Junction Detected

3. Import the modules in the main file and run the program.

Sample Input
001100
Expected Output
Sensor Values: [0, 0, 1, 1, 0, 0]
Active Sensors: 2
Robot Action: Move Forward